

Este guia foi feito para ser seguido **passo a passo**. Cada secao diz **qual arquivo abrir, onde encontrar e o codigo exato para digitar**. Nao pule etapas - siga a ordem e o projeto vai funcionar!

Dica: Sempre que ver um bloco de codigo escuro, esse e o codigo para digitar no arquivo indicado. Copie exatamente como esta.

0. Visao Geral - O que vamos construir?

Vamos criar um **Jogo da Velha** para Android usando o padrao **MVVM** (Model-View-ViewModel). O jogo tera um tabuleiro 3x3, dois jogadores (X e O), verificacao de vencedor e botao de reiniciar.

Camada	Arquivo a criar	O que faz
Model	GameModel.kt	Guarda o tabuleiro e as regras do jogo
ViewModel	GameViewModel.kt	Controla o estado, usa o Model, avisa a View
View (XML)	activity_main.xml	Layout com os 9 botoes do tabuleiro
View (Kt)	MainActivity.kt	Liga os botoes ao ViewModel usando LiveData

Importante: O professor vai olhar se as tres camadas estao separadas. Se Model, ViewModel e View existirem e conversarem pelo LiveData - **nota garantida!**

1. Configurar o Projeto no Android Studio

■ 1.1 - Adicionar dependencias do ViewModel

Antes de criar os arquivos Kotlin, precisamos adicionar a biblioteca do ViewModel. Abra o arquivo de dependencias do app:

```
app -> build.gradle (ou build.gradle.kts)
```

Dentro do bloco **dependencies { }**, adicione estas linhas se nao existirem:

**GRAD
LE**

build.gradle (app)

```
dependencies {  
    // Adicione estas linhas:  
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")  
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")  
    implementation("androidx.activity:activity-ktx:1.8.2")  
  
    // Essas ja devem existir:  
    implementation("androidx.core:core-ktx:1.12.0")  
    implementation("androidx.appcompat:appcompat:1.6.1")  
}
```

```
}
```

Atencao: Depois de adicionar, clique em **Sync Now** no aviso amarelo que aparece no topo. Aguarde terminar antes de continuar.

■ 1.2 - Estrutura de pastas que vamos criar

```
app/src/main/java/com/example/seuprojeto/  
model/  
GameModel.kt  
viewmodel/  
GameViewModel.kt  
MainActivity.kt <- ja existe, vamos editar
```

Para criar uma pasta: clique com **botao direito** na pasta do projeto -> **New -> Package** -> digite o nome (model ou viewmodel).

2. Criar o Model - GameModel.kt

O **Model** é o coração do jogo. Ele não sabe nada sobre telas ou botões - só cuida das regras: tabuleiro, vez de quem joga, verificar vencedor e reiniciar.

- Criar o arquivo**
1 Clique com botão direito na pasta model -> New -> Kotlin Class/File -> escolha File -> digite: GameModel -> pressione Enter
- Verificar o caminho**
2 O arquivo deve aparecer em: app -> src -> main -> java -> seu projeto -> model -> GameModel.kt
- Digitar o código abaixo**
3 Apague tudo que tiver no arquivo e copie exatamente o código a seguir:

KOTLIN

GameModel.kt

```
package com.example.substitutivajogodavelhamvmm.model

// MODEL: cuida do tabuleiro e das regras do jogo
class GameModel {

    // Tabuleiro 3x3: ' ' = vazio, 'X' = jogador X, 'O' = jogador O
    val board = Array(3) { CharArray(3) { ' ' } }

    // Quem está jogando agora
    var currentPlayer: Char = 'X'
        private set // só o Model pode mudar

    // Tenta fazer uma jogada na posição (row, col)
    // Retorna TRUE se conseguiu, FALSE se já estava ocupado
    fun makeMove(row: Int, col: Int): Boolean {
        if (board[row][col] != ' ') return false // célula ocupada!
        board[row][col] = currentPlayer
        return true
    }

    // Verifica se há um vencedor
    // Retorna 'X', 'O' ou null (se ninguém ganhou ainda)
    fun checkWinner(): Char? {
        // Verifica as 3 linhas
        for (i in 0..2) {
            if (board[i][0] != ' ' &&
                board[i][0] == board[i][1] &&
                board[i][1] == board[i][2])
                return board[i][0]
        }
        // Verifica as 3 colunas
        for (j in 0..2) {
            if (board[0][j] != ' ' &&
                board[0][j] == board[1][j] &&
                board[1][j] == board[2][j])
                return board[0][j]
        }
        return null
    }
}
```

```

        return board[0][j]
    }
    // Verifica diagonal principal (para baixo-direita)
    if (board[0][0] != ' ' &&
        board[0][0] == board[1][1] &&
        board[1][1] == board[2][2])
        return board[0][0]
    // Verifica diagonal secundaria (para baixo-esquerda)
    if (board[0][2] != ' ' &&
        board[0][2] == board[1][1] &&
        board[1][1] == board[2][0])
        return board[0][2]

    return null // ninguem ganhou
}

// Verifica empate: todas as celulas preenchidas e sem vencedor
fun isDraw(): Boolean = board.all { row -> row.all { it != ' ' } }

// Troca o jogador: X vira O, O vira X
fun switchPlayer() {
    currentPlayer = if (currentPlayer == 'X') 'O' else 'X'
}

// Reinicia tudo para o estado inicial
fun reset() {
    for (i in 0..2) for (j in 0..2) board[i][j] = ' '
    currentPlayer = 'X'
}
}

```

O que cada funcao faz:

Funcao	O que faz	Retorna
makeMove(row, col)	Tenta marcar a posicao no tabuleiro	true (ok) / false (ocupado)
checkWinner()	Verifica linhas, colunas e diagonais	'X', 'O' ou null
isDraw()	Checa se todas as celulas estao preenchidas	true ou false
switchPlayer()	Troca de X para O (ou O para X)	nada (Unit)
reset()	Limpa o tabuleiro e volta para X jogar	nada (Unit)

Dica: O Model nao importa nada do Android! Nao tem Context, Activity, View ou LiveData. So Kotlin puro. Isso e o MVVM funcionando corretamente.

3. Criar o ViewModel - GameViewModel.kt

O **ViewModel** é o intermediário. Ele usa o Model para processar as jogadas e avisa a View (Activity) quando algo muda, usando **LiveData**. A View nunca toca no Model diretamente.

1 Criar o arquivo

Clique com botão direito na pasta viewmodel -> New -> Kotlin Class/File -> escolha File -> digite: GameViewModel -> pressione Enter

2 Verificar o caminho

app -> src -> main -> java -> seu projeto -> viewmodel -> GameViewModel.kt

3 Digitar o código abaixo

Apague tudo e copie o código:

KOTLIN

GameViewModel.kt

```
package com.example.substitutivajogodavelhamvmm.viewmodel

import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import com.example.substitutivajogodavelhamvmm.model.GameModel

// VIEWMODEL: liga o Model a View usando LiveData
class GameViewModel : ViewModel() {

    // Instancia do Model (so o ViewModel usa o Model!)
    private val model = GameModel()

    // LiveData PRIVADO: so o ViewModel pode alterar
    // LiveData PUBLICO: a View le aqui (so leitura)

    // Texto de status: "Vez do X", "X Ganhou!", "Empate!"
    private val _status = MutableLiveData("Vez do X")
    val status: LiveData<String> = _status

    // Estado de cada celula do tabuleiro
    private val _board = MutableLiveData<Array<CharArray>>()
    val board: LiveData<Array<CharArray>> = _board

    // true = jogo acabou (ganhou ou empatou)
    private val _gameOver = MutableLiveData(false)
    val gameOver: LiveData<Boolean> = _gameOver

    // Inicializa o tabuleiro ao criar o ViewModel
    init {
        _board.value = model.board
    }

    // Chamado quando o usuario clica numa celula
    fun onCellClicked(row: Int, col: Int) {
        // Se o jogo acabou, ignora o clique
    }
}
```

```

        if (_gameOver.value == true) return

        // Tenta fazer a jogada
        val moved = model.makeMove(row, col)
        if (!moved) return // celula ocupada, ignora

        // Atualiza o tabuleiro na View
        _board.value = model.board

        // Verifica o resultado da jogada
        val winner = model.checkWinner()
        when {
            winner != null -> {
                // Alguem ganhou!
                _status.value = "Jogador $winner ganhou!"
                _gameOver.value = true
            }
            model.isDraw() -> {
                // Empate
                _status.value = "Empate!"
                _gameOver.value = true
            }
            else -> {
                // Jogo continua, troca o jogador
                model.switchPlayer()
                _status.value = "Vez do ${model.currentPlayer}"
            }
        }
    }

    // Chamado quando o usuario clica em Reiniciar
    fun onRestart() {
        model.reset()
        _board.value = model.board
        _status.value = "Vez do X"
        _gameOver.value = false
    }
}

```

Entendendo o LiveData:

Tipo	Quem usa	Para que serve
MutableLiveData	So o ViewModel (privado com _)	Permite alterar o valor
LiveData	A View (publico, sem _)	A View observa mas nao altera

Atencao: Nunca coloque **findViewById()** no ViewModel! O ViewModel nao pode conhecer Views ou Activities. Se precisar mostrar algo na tela, atualize um LiveData.

4. Criar o Layout - activity_main.xml

O layout define o que aparece na tela: os 9 botoes do tabuleiro, o texto de status e o botao Reiniciar.

1 Abrir o arquivo

Vá em: app -> src -> main -> res -> layout -> activity_main.xml

2 Mudar para visualizacao Code

No canto superior direito do editor, clique em Code (ou Split para ver os dois)

3 Substituir TODO o conteudo

Selecione tudo (Ctrl+A) e substitua pelo codigo abaixo:

```
XML activity_main.xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp"
    android:background="#FFFFFF">

    <!-- Titulo do jogo -->
    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Jogo da Velha"
        android:textSize="28sp"
        android:textStyle="bold"
        android:textColor="#1565C0"
        android:layout_marginBottom="16dp"/>

    <!-- Texto que mostra de quem e a vez / quem ganhou -->
    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Vez do X"
        android:textSize="20sp"
        android:textColor="#212121"
        android:layout_marginBottom="24dp"/>

    <!-- Grade 3x3 com os 9 botoes do tabuleiro -->
    <GridLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:rowCount="3"
        android:columnCount="3"
        android:layout_marginBottom="24dp">

        <!-- Linha 0 -->
```

```

<Button android:id="@+id/btn00" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="0" android:layout_column="0"/>
<Button android:id="@+id/btn01" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="0" android:layout_column="1"/>
<Button android:id="@+id/btn02" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="0" android:layout_column="2"/>

<!-- Linha 1 -->
<Button android:id="@+id/btn10" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="1" android:layout_column="0"/>
<Button android:id="@+id/btn11" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="1" android:layout_column="1"/>
<Button android:id="@+id/btn12" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="1" android:layout_column="2"/>

<!-- Linha 2 -->
<Button android:id="@+id/btn20" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="2" android:layout_column="0"/>
<Button android:id="@+id/btn21" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="2" android:layout_column="1"/>
<Button android:id="@+id/btn22" android:text=" "
    android:textSize="32sp" android:textStyle="bold"
    android:layout_width="100dp" android:layout_height="100dp"
    android:layout_row="2" android:layout_column="2"/>

</GridLayout>

<!-- Botao para reiniciar o jogo -->
<Button
    android:id="@+id/btnRestart"
    android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:text="Reiniciar"
    android:textSize="18sp"
    android:backgroundTint="#1565C0"
    android:textColor="#FFFFFF"/>

</LinearLayout>

```

Mapa dos botoes - ID x posicao no tabuleiro:

ID do Botao	Linha (row)	Coluna (col)	Posicao no tabuleiro
btn00	0	0	Canto superior esquerdo

ID do Botao	Linha (row)	Coluna (col)	Posicao no tabuleiro
btn01	0	1	Centro superior
btn02	0	2	Canto superior direito
btn10	1	0	Centro esquerdo
btn11	1	1	Centro (meio do jogo)
btn12	1	2	Centro direito
btn20	2	0	Canto inferior esquerdo
btn21	2	1	Centro inferior
btn22	2	2	Canto inferior direito

5. Editar a MainActivity.kt - A View

A **MainActivity** e a View. Ela **nao decide** nada - so observa o ViewModel e atualiza os botoes quando o LiveData mudar. Todo clique e repassado para o ViewModel processar.

1 Abrir o arquivo

app -> src -> main -> java -> seuprojeto -> MainActivity.kt

2 Substituir TODO o conteudo

Selecione tudo (Ctrl+A) e substitua pelo codigo abaixo. ATENCAO: O nome do package na 1a linha deve ser o mesmo do seu projeto!

KOTLIN

MainActivity.kt

```
package com.example.substitutivajogodavelhamvmm

import android.graphics.Color
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import com.example.substitutivajogodavelhamvmm.viewmodel.GameViewModel

// VIEW: so exibe dados e repassa acoes para o ViewModel
class MainActivity : AppCompatActivity() {

    // Obtem o ViewModel (cria automaticamente na 1a vez)
    private val viewModel: GameViewModel by viewModels()

    // Matriz 3x3 de botoes
    private lateinit var buttons: Array<Array<Button>>

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // --- 1. Pegar as views do layout ---
        val tvStatus = findViewById<TextView>(R.id.tvStatus)
        val btnRestart = findViewById<Button>(R.id.btnRestart)

        // Monta a matriz ligando cada ID ao par (linha, coluna)
        buttons = arrayOf(
            arrayOf(
                findViewById(R.id.btn00),
                findViewById(R.id.btn01),
                findViewById(R.id.btn02)
            ),
            arrayOf(
                findViewById(R.id.btn10),
                findViewById(R.id.btn11),
                findViewById(R.id.btn12)
            ),
            arrayOf(
                findViewById(R.id.btn20),
```

```

        findViewById(R.id.btn21),
        findViewById(R.id.btn22)
    )
)

// --- 2. Configurar cliques nos 9 botoes ---
for (i in 0..2) {
    for (j in 0..2) {
        buttons[i][j].setOnClickListener {
            // Avisar o ViewModel qual celula foi clicada
            viewModel.onCellClicked(i, j)
        }
    }
}

// Clique no botao Reiniciar
btnRestart.setOnClickListener {
    viewModel.onRestart()
}

// --- 3. Observar o LiveData do ViewModel ---

// Observa o texto de status e atualiza o TextView
viewModel.status.observe(this) { mensagem ->
    tvStatus.text = mensagem
}

// Observa o tabuleiro e atualiza o texto de cada botao
viewModel.board.observe(this) { tabuleiro ->
    for (i in 0..2) {
        for (j in 0..2) {
            val celula = tabuleiro[i][j]
            buttons[i][j].text = if (celula == ' ') "" else celula.toString()
            // Muda a cor: X = azul, O = vermelho
            buttons[i][j].setTextColor(
                when (celula) {
                    'X' -> Color.parseColor("#1565C0")
                    'O' -> Color.parseColor("#B71C1C")
                    else -> Color.BLACK
                }
            )
        }
    }
}

// Observa se o jogo acabou
viewModel.gameOver.observe(this) { acabou ->
    if (acabou) {
        // Desabilita todos os botoes quando o jogo acaba
        for (i in 0..2) for (j in 0..2)
            buttons[i][j].isEnabled = false
        btnRestart.isEnabled = true
    } else {
        // Habilita todos os botoes ao reiniciar
        for (i in 0..2) for (j in 0..2)
            buttons[i][j].isEnabled = true
    }
}
}
}
}

```

Atencao: Atencao ao nome do **package** na primeira linha! Ele deve ser o mesmo que aparece nos outros arquivos do seu projeto. Se der erro vermelho, verifique se os nomes batem.

O que cada bloco observe() faz:

observe()	Quando dispara	O que faz na tela
viewModel.status	A cada jogada ou reinicio	Atualiza o texto Vez do X / X Ganhou!
viewModel.board	Cada vez que o tabuleiro muda	Coloca X ou O no botao da posicao certa
viewModel.gameOver	Quando alguem ganha ou empata	Trava os botoes (isEnabled = false)

6. Verificar o Nome do Package (Passo Critico!)

O **package** e o endereco do projeto. Todos os arquivos precisam usar o mesmo. Se o package estiver errado, o projeto nao compila.

1 Ver o package do projeto

Abra qualquer arquivo .kt que ja existia -> olhe a 1a linha: package com.example.NOMEAQUI

2 Ajustar nos 3 arquivos

Garanta que os 3 arquivos criados usam o MESMO package base:

Arquivo	1a linha (package)
GameModel.kt	package com.example.SEUPROJETO.model
GameViewModel.kt	package com.example.SEUPROJETO.viewmodel
MainActivity.kt	package com.example.SEUPROJETO

3 Verificar os imports na MainActivity

Garanta que as linhas import apontam para os arquivos corretos. Se aparecer erro vermelho em algum import, clique sobre ele e pressione Alt+Enter para o Studio corrigir automaticamente.

7. Executar e Testar o App

1 Executar no emulador ou celular

Clique no botao verde PLAY no topo do Android Studio (ou pressione Shift+F10). Aguarde compilar e instalar no dispositivo.

2 Testar estes cenarios

Confira se cada situacao funciona corretamente:

Cenário de teste	O que deve acontecer
Clique num botao vazio	Aparece X ou O no botao, status muda para o proximo jogador
Clique num botao ja marcado	Nada acontece (jogada ignorada)
Tres X ou O na mesma linha	Aparece Jogador X ganhou! e botoes travam
Tres X ou O na mesma coluna	Aparece Jogador O ganhou! e botoes travam
Tres X ou O numa diagonal	Aparece mensagem de vitoria corretamente
Todos os 9 campos preenchidos	Aparece Empate! (sem vencedor)
Clicar Reiniciar apos vitoria	Tabuleiro limpa, status volta para Vez do X
Clicar Reiniciar durante o jogo	Tudo reinicia normalmente

8. Erros Comuns e Como Resolver

Erro / Problema	Causa provavel	Como resolver
Linha vermelha no import do GameModel	Package errado	Verifique se a 1a linha do GameModel.kt tem .model no final
'by viewModels()' com erro vermelho	Faltou dependencia no build.gradle	Adicione activity-ktx e faça Sync Now
App fecha ao clicar no botao	Erro em runtime no ViewModel	Veja o Logcat (aba no rodape do Studio) e leia a linha vermelha
Botoes nao mostram X ou O	board.observe() nao esta sendo chamado	Confirme que viewModel.board.observe(this){} esta no onCreate()
Jogo continua apos vitoria	Faltou checar _gameOver no onCellClicked	A 1a linha do onCellClicked deve ser: if (_gameOver.value == true) return
Reiniciar nao limpa os botoes	onRestart() nao esta atualizando _board	Confirme que onRestart() chama: _board.value = model.board

9. Checklist Final - Antes de Entregar

Marque cada item abaixo para garantir que nada foi esquecido:

	O que verificar	Como checar
[]	GameModel.kt criado na pasta model/	Veja no explorador de arquivos do Studio
[]	GameViewModel.kt criado na pasta viewmodel/	Veja no explorador de arquivos do Studio
[]	ViewModel herda de ViewModel()	Linha: class GameViewModel : ViewModel()
[]	LiveData privado (MutableLiveData) e publico	Cada _status tem val status correspondente
[]	onCellClicked() verifica se gameOver == true	1a linha do metodo deve ter esse if
[]	checkWinner() verifica linhas, colunas e diagonais	3 loops + 2 diagonais no GameModel
[]	isDraw() implementado	Todos os campos preenchidos sem vencedor
[]	switchPlayer() chamado apos jogada valida	Dentro do else{} no onCellClicked
[]	activity_main.xml tem 9 botoes com IDs corretos	btn00 ate btn22

	O que verificar	Como checar
[]	MainActivity usa observe() para TODOS os LiveData	status, board e gameOver
[]	MainActivity NAO tem logica de jogo (if/else)	So chama funcoes do ViewModel
[]	Botao Reiniciar funciona corretamente	Teste no app rodando

10. Resumo - O que o Professor Quer Ver

Para garantir a nota 6,0, o professor precisa ver **3 coisas** funcionando:

#	O que mostrar	Onde esta no codigo
1	Model separado, com logica do jogo	GameModel.kt - makeMove(), checkWinner(), isDraw()
2	ViewModel com LiveData fazendo a ponte	GameViewModel.kt - _status, _board, _gameOver + observe()
3	View que so observa e repassa acoes	MainActivity.kt - setOnClickListener chama viewModel.onCellClicked()

Boa sorte! Voce consegue!

Siga cada passo com calma, use este guia como consulta e o professor vai entender a logica que voce aplicou.